

Continual Learning sobre Seq-CIFAR-10: Implementación y comparación de métodos para mitigar el Catastrophic Forgetting

Giacometti, Mateo
Ingeniería en Inteligencia Artificial
Universidad de San Andrés
Buenos Aires, Argentina
mgiacometti@udesa.edu.ar

Martín Bernal, Tiziano Leví
Ingeniería en Inteligencia Artificial
Universidad de San Andrés
Buenos Aires, Argentina
tmartinbernal@udesa.edu.ar

I. INTRODUCCIÓN

El **aprendizaje continuo** (*continual learning*) es un paradigma de *machine learning* orientado a entrenar modelos capaces de incorporar nuevo conocimiento de manera secuencial, a medida que se dispone de datos o tareas adicionales. A diferencia del aprendizaje supervisado tradicional, en el que el modelo se entrena una única vez sobre un conjunto de datos estático, en aprendizaje continuo las tareas se presentan una tras otra y, en general, no se conserva acceso completo a los datos de etapas anteriores. El objetivo es construir un único modelo que mantenga un buen desempeño sobre todas las tareas observadas hasta el momento. Sin embargo, esta capacidad de adaptación plantea un desafío central: el **olvido catastrófico** (*catastrophic forgetting*). Cuando el modelo se actualiza para aprender una nueva tarea, sus parámetros pueden ajustarse de forma tal que deterioren el conocimiento adquirido previamente, reduciendo de manera significativa el rendimiento sobre tareas ya aprendidas. Por este motivo, gran parte de la investigación en aprendizaje continuo se enfoca en diseñar estrategias que permitan equilibrar la capacidad de aprender información nueva (*plasticity*) y la capacidad de preservar conocimiento previo (*stability*).

En este trabajo práctico se estudia el problema del aprendizaje continuo en clasificación de imágenes utilizando el dataset *CIFAR-10*, organizado como una secuencia de tareas (*Seq-CIFAR-10*). El objetivo es comparar distintas estrategias para mitigar el olvido catastrófico a medida que el modelo se entrena secuencialmente sobre subconjuntos de clases. Se analizan enfoques que van desde un baseline basado en fine-tuning hasta métodos más avanzados, incluyendo regularización de parámetros, destilación de conocimiento y técnicas recientes basadas en aprendizaje contrastivo. Además, se consideran distintos escenarios de evaluación que reflejan diferentes niveles de información disponible durante la inferencia, con el fin de comparar el desempeño de cada método.

En las siguientes secciones se presenta la metodología adoptada, incluyendo el preprocesamiento y la organización del conjunto de datos, las arquitecturas empleadas y la descripción

de los métodos implementados, junto con los criterios de entrenamiento y evaluación utilizados. Finalmente, se reportan los resultados obtenidos y se discuten de manera comparativa, extrayendo las conclusiones principales del estudio.

II. METODOLOGÍA

II-A. Manipulación de datos

II-A1. Preparación del dataset

Como se estipuló en la introducción, este trabajo empleó el **dataset** denominado *Seq-CIFAR-10*, una adaptación de *CIFAR-10* al escenario de aprendizaje continuo. El dataset original contiene 60000 imágenes en color de 32×32 píxeles, distribuidas en 10 clases (50000 de entrenamiento y 10000 de test). Para construir *Seq-CIFAR-10*, las clases se dividen en 5 tareas secuenciales disjuntas, cada una compuesta por 2 clases.

Tabla I
ESTRUCTURA DE *Seq-CIFAR-10*

Task	Classes	Train Set	Test Set
0	'airplane', 'automobile'	10000	2000
1	'bird', 'cat'	10000	2000
2	'deer', 'dog'	10000	2000
3	'frog', 'horse'	10000	2000
4	'ship', 'truck'	10000	2000

Mediante esta estructuración de los datos se garantiza una distribución balanceada entre las clases en cada tarea, evitando sesgos que puedan afectar el proceso de entrenamiento y la evaluación del modelo.

II-A2. Data Loaders

A la hora de trabajar con aprendizaje continuo, la forma en que se presentan los datos durante el entrenamiento es tan importante como la arquitectura o la función de pérdida. El **data loader** no solo carga imágenes, sino que define qué muestras ve el modelo, en qué orden y bajo qué transformaciones. En particular, garantiza que durante el entrenamiento de la tarea t no se utilicen muestras de las tareas $1, \dots, t-1$, respetando la restricción central del aprendizaje continuo.

En este trabajo se implementaron dos tipos de loaders según la etapa de entrenamiento, cada uno con transformaciones

distintas. Para las tareas basadas en aprendizaje contrastivo se utilizó un loader con *TwoCropTransform*, que genera dos vistas aumentadas independientes por imagen. Cada vista aplica secuencialmente *RandomResizedCrop* (escala $[0.2, 1.0]$), *RandomHorizontalFlip* ($p = 0.5$), *ColorJitter* ($p = 0.8$), y *RandomGrayscale* ($p = 0.2$), seguido de normalización con la media y desvío estándar de *CIFAR-10*. Este pipeline busca producir vistas suficientemente diferentes para que la loss contrastiva aporte una señal informativa y no trivial.

Por otro lado, para evaluación y entrenamiento supervisado se utilizó una transformación estándar sin aumentación, compuesta únicamente por conversión a tensor y normalización, evitando perturbaciones que puedan sesgar la medición del desempeño.

II-A3. Replay Buffer

El olvido catastrófico ocurre porque, al entrenar sobre la tarea t , los gradientes actualizan los pesos para minimizar el error en las clases nuevas sin preservar el conocimiento adquirido en tareas previas. Una estrategia simple para mitigar este fenómeno es almacenar un subconjunto de muestras anteriores y reintroducirlas durante el entrenamiento, mecanismo conocido como *experience replay*, implementado mediante un **replay buffer**. Dado que el buffer tiene capacidad fija M , es necesario definir una política de selección que evite sesgos hacia tareas recientes. En este trabajo se utilizó *reservoir sampling*, que garantiza que tras observar N muestras, cada una tenga probabilidad M/N de permanecer en el buffer. El proceso es online: si el buffer no está lleno, la muestra se agrega; caso contrario, se inserta con probabilidad M/n reemplazando una entrada aleatoria. Esto permite mantener una muestra uniforme de todos los datos vistos de manera eficiente. Siguiendo [2], se utilizó un buffer de tamaño $M = 200$. Las imágenes se almacenan normalizadas y sin aumentación, aplicando las transformaciones contrastivas únicamente al momento de muestreo durante el entrenamiento.

II-B. Pre-entrenamiento con Aprendizaje Contrastivo Supervisado

El primer paso del pipeline de entrenamiento consiste en pre-entrenar el backbone utilizando **Aprendizaje Contrastivo Supervisado** (*Supervised Contrastive Learning*, SupCon), propuesto por Khosla et al. [1]. A diferencia del aprendizaje contrastivo auto-supervisado, que define los pares positivos únicamente a partir de aumentaciones de la misma imagen, SupCon aprovecha las etiquetas de clase para extender la noción de positividad: todas las muestras de la misma clase en el batch son consideradas positivas entre sí, independientemente de si provienen de la misma imagen original. Esto permite una señal de entrenamiento más rica y representaciones más estructuradas en el espacio de embeddings. Formalmente, dado un batch de N muestras, se generan $2N$ vistas aumentadas mediante *TwoCropTransform* (descrito en la sección II-A2). Cada vista $\tilde{x}_i$ es mapeada a un vector unitario $z_i = (g \circ f)_\psi(\tilde{x}_i)$ mediante la composición del encoder f_θ y el *projection head* g_ϕ . La función de pérdida SupCon se define entonces como:

$$\mathcal{L}^{\text{sup}} = \sum_{i=1}^{2N} \frac{-1}{|\mathcal{P}_i|} \sum_{p \in \mathcal{P}_i} \log \frac{\exp(z_i \cdot z_p / \tau)}{\sum_{k \neq i} \exp(z_i \cdot z_k / \tau)} \quad (1)$$

donde $\tau > 0$ es un *hiperparámetro de temperatura* y $\mathcal{P}_i = \{j \neq i : y_j = y_i\}$ es el conjunto de índices de muestras positivas respecto al ancla $\tilde{x}_i$. El denominador suma sobre todos los elementos del batch excepto el ancla, de modo que la loss empuja las representaciones de muestras de la misma clase a acercarse entre sí, mientras aleja las de clases distintas.

La *arquitectura* empleada sigue la propuesta del paper de referencia en el que se basa este trabajo [2]: un *ResNet-18* [3] adaptado para imágenes de 32×32 píxeles (reemplazando la convolución inicial de 7×7 por una de 3×3 y eliminando el *max pooling*) actúa como encoder f_θ , cuya salida es un vector de 512 dimensiones. Sobre este se monta un *projection head* g_ϕ compuesto por dos capas lineales con una capa oculta de 512 unidades, *Batch Normalization* y activación *ReLU*, que proyecta las representaciones a un espacio de 128 dimensiones donde se computa la loss. Cabe destacar que el *projection head* se utiliza exclusivamente durante el pre-entrenamiento contrastivo: en la evaluación y en las etapas de aprendizaje continuo se descarta, y se trabaja directamente con las representaciones del encoder f_θ .

El pre-entrenamiento se realiza únicamente sobre los datos de la *Task 0*, durante 50 epoch y, una vez finalizado el pre-entrenamiento, se congela el encoder y se entrena una *cabeza de clasificación lineal* sobre la *Task 0* durante 10 epoch. La elección de una cabeza de clasificación lineal no es arbitraria: es una práctica estándar en aprendizaje por representaciones para evaluar la calidad de los embeddings aprendidos de forma aislada. Si el clasificador fuera no lineal, parte del trabajo de separar las clases podría recaer en las capas adicionales del clasificador en lugar del encoder, enmascarando la verdadera calidad de las representaciones. Al restringir el clasificador a una única capa lineal y congelar el backbone, la accuracy obtenida refleja directamente cuán linealmente separables son los embeddings producidos por f_θ . Esto es una propiedad deseable en el contexto de aprendizaje continuo, donde el encoder debe transferirse a tareas futuras sin ser reentrenado desde cero.

II-C. Métodos de Aprendizaje Continuo

Una vez obtenido el backbone pre-entrenado, tal como se describió en la sección II-B, este se utiliza como punto de partida para aprender de manera secuencial las tareas restantes. Sobre esta base, se implementan y comparan los distintos métodos de aprendizaje continuo que se describen a continuación, con el objetivo de analizar su capacidad para incorporar nuevo conocimiento sin deteriorar significativamente el desempeño sobre las tareas previamente aprendidas.

II-C1. Fine-Tuning Naive (Baseline)

Este primer método consiste en un *re-entrenamiento directo* del modelo con *entropía cruzada estándar* en cada tarea nueva, sin ningún mecanismo para preservar el conocimiento adquirido previamente. En cada tarea t , todos los parámetros del

modelo se actualizan exclusivamente en función de los datos de esa tarea, ignorando por completo las tareas $1, \dots, t-1$.

La función de pérdida se reduce entonces a:

$$\mathcal{L}_t(\theta) = -\frac{1}{|\mathcal{D}_t|} \sum_{(x,y) \in \mathcal{D}_t} \log p_\theta(y | x) \quad (2)$$

donde $\mathcal{D}_t$ es el conjunto de entrenamiento de la tarea t y $p_\theta(y | x)$ es la distribución de salida del modelo parametrizado por θ .

Este enfoque no constituye un método de aprendizaje continuo en sentido estricto, sino una *cota inferior* de referencia. Su propósito es cuantificar el olvido catastrófico en su forma más pura: al no existir ninguna restricción sobre las actualizaciones del gradiente, los pesos del modelo convergen hacia un mínimo de la loss de la tarea actual, sobrescribiendo las representaciones que eran útiles para tareas anteriores. El deterioro en el rendimiento sobre tareas pasadas que exhibe este método establece el piso a partir del cual se mide la mejora aportada por cada uno de los métodos subsiguientes.

II-C2. Elastic Weight Consolidation (EWC)

El segundo método implementado es **Elastic Weight Consolidation (EWC)**, propuesto por Kirkpatrick et al. [4]. *EWC* pertenece a la familia de métodos de *regularización de parámetros*, cuya idea central es restringir las actualizaciones del gradiente para que los parámetros importantes para tareas anteriores no se modifiquen drásticamente al aprender una nueva tarea. A diferencia del *fine-tuning naive* visto en la sección II-C1, *EWC* no requiere almacenar datos de tareas pasadas ni modificar la arquitectura del modelo; incorpora el conocimiento previo directamente en la función de pérdida mediante un término de penalización.

La motivación conceptual parte de una observación sobre la geometría del espacio de parámetros: no todos los pesos del modelo contribuyen de igual forma al rendimiento en una tarea dada. Algunos parámetros son críticos (pequeñas perturbaciones en su valor degradan significativamente la loss) mientras que otros son relativamente irrelevantes y pueden modificarse libremente sin afectar el desempeño. *EWC* formaliza esta intuición utilizando la *matriz de información de Fisher*, que cuantifica la sensibilidad de la distribución de salida del modelo ante perturbaciones en cada parámetro. Dado que calcular y almacenar la matriz de Fisher completa es computacionalmente prohibitivo para redes profundas, *EWC* utiliza su diagonal F_i que puede interpretarse como una medida de la importancia del parámetro θ_i para la tarea anterior.

Formalmente, al entrenar la tarea t , *EWC* añade a la loss de *entropía cruzada estándar* un término de *penalización cuadrática* sobre los parámetros que fueron importantes para la tarea $t-1$:

$$\mathcal{L}_t(\theta) = \mathcal{L}_t^{\text{CE}}(\theta) + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^*)^2 \quad (3)$$

donde θ^* son los parámetros óptimos al finalizar el entrenamiento de la tarea anterior, F_i es el i -ésimo elemento de la

diagonal de la matriz de Fisher estimada sobre esos datos, y λ es un hiperparámetro que controla el balance entre aprender la nueva tarea y preservar el conocimiento previo. El término de penalización actúa como un resorte elástico (de ahí el nombre del método) que atrae los parámetros hacia θ^* con una fuerza proporcional a su importancia F_i los parámetros críticos para la tarea anterior son fuertemente penalizados si se alejan de sus valores previos, mientras que los parámetros poco importantes pueden modificarse con relativa libertad.

La diagonal de Fisher se estima empíricamente al finalizar el entrenamiento de cada tarea, promediando el cuadrado de los gradientes de la log-verosimilitud sobre un subconjunto de los datos de esa tarea:

$$F_i \approx \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} \left(\frac{\partial \log p_\theta(y | x)}{\partial \theta_i} \right)^2 \quad (4)$$

Desde la perspectiva del balance *stability-plasticity*, *EWC* ofrece un mecanismo de control explícito a través de λ : valores altos priorizan la estabilidad del conocimiento previo a costa de limitar la capacidad de adaptación a nuevas tareas, mientras que valores bajos permiten mayor plasticidad con mayor riesgo de olvido.

Una limitación conocida de *EWC* es que, en su formulación original, solo penaliza las desviaciones respecto a los parámetros de la tarea inmediatamente anterior. En un escenario con múltiples tareas, esto puede resultar insuficiente si las tareas sucesivas empujan los parámetros en direcciones conflictivas, acumulando olvido de forma progresiva a pesar de la penalización. En la implementación utilizada se mitiga este problema acumulando un término de penalización independiente por cada tarea aprendida, de modo que la loss al entrenar la tarea t es:

$$\mathcal{L}_t(\theta) = \mathcal{L}_t^{\text{CE}}(\theta) + \frac{\lambda}{2} \sum_{k < t} \sum_i F_i^{(k)} (\theta_i - \theta_i^{*(k)})^2 \quad (5)$$

donde $F_i^{(k)}$ y $\theta^{*(k)}$ son la diagonal de Fisher y los parámetros óptimos estimados al finalizar la tarea k respectivamente.

II-C3. Learning without Forgetting (LwF)

El tercer método implementado es **Learning without Forgetting (LwF)**, propuesto por Li y Hoiem [5]. *LwF* pertenece a la familia de métodos basados en destilación de conocimiento (*knowledge distillation*), y comparte con *EWC* el objetivo de preservar el conocimiento previo sin almacenar datos de tareas anteriores. Sin embargo, la estrategia es fundamentalmente distinta: en lugar de penalizar las desviaciones en el espacio de parámetros, *LwF* regula el comportamiento funcional del modelo, preservando las predicciones que el modelo realizaba sobre las clases anteriores.

La idea central es la siguiente. Al comenzar el entrenamiento de la tarea t , se guarda una copia del modelo actual como modelo de referencia θ^* , que representa el conocimiento acumulado hasta ese momento. Durante el entrenamiento de la nueva tarea, el modelo actualizado θ debe cumplir dos objetivos simultáneamente: aprender a clasificar correctamente

las clases nuevas mediante entropía cruzada estándar, y mantener distribuciones de salida similares a las del modelo de referencia sobre las clases anteriores. Este segundo objetivo se implementa mediante una loss de destilación que utiliza las predicciones suavizadas del modelo de referencia como *soft targets*.

Formalmente, la función de pérdida de *LwF* se define como:

$$\mathcal{L}_t(\theta) = \mathcal{L}_t^{\text{CE}}(\theta) + \alpha \cdot \mathcal{L}^{\text{distill}}(\theta, \theta^*) \quad (6)$$

donde $\mathcal{L}_t^{\text{CE}}$ es la entropía cruzada sobre las clases de la tarea actual, α es un hiperparámetro que pondera la importancia relativa de ambos objetivos, y $\mathcal{L}^{\text{distill}}$ es la loss de destilación, definida como la entropía cruzada entre las distribuciones suavizadas del modelo de referencia y del modelo actual sobre las clases anteriores:

$$\mathcal{L}^{\text{distill}}(\theta, \theta^*) = - \sum_{c \in \mathcal{C}_{\text{old}}} p_c^* \log p_c \quad (7)$$

donde $p_c = \text{softmax}(z_c/T)$ y $p_c^* = \text{softmax}(z_c^*/T)$ son las probabilidades suavizadas por temperatura T del modelo actual y de referencia respectivamente, y $\mathcal{C}_{\text{old}}$ es el conjunto de clases de tareas anteriores. Dado que $H(p^*) = - \sum_c p_c^* \log p_c^*$ es constante respecto a θ , minimizar esta expresión equivale a minimizar la divergencia KL entre ambas distribuciones, lo que empuja al modelo actual a replicar el comportamiento del modelo de referencia sobre las clases previas.

En la práctica, los logits se dividen por T antes de aplicar la softmax, lo que reduce la magnitud de los gradientes de destilación por un factor $1/T^2$ respecto a los gradientes de la entropía cruzada estándar. Para compensar este efecto y mantener el balance implícito entre ambos términos independientemente de la temperatura elegida, la loss de destilación se reescala por T^2 :

$$\mathcal{L}^{\text{distill}}(\theta, \theta^*) = -T^2 \sum_{c \in \mathcal{C}_{\text{old}}} p_c^* \log p_c \quad (8)$$

La temperatura $T > 1$ suaviza las distribuciones de salida, amplificando las diferencias entre clases poco probables y permitiendo que el modelo capture relaciones de similitud entre clases que una distribución *one-hot* estándar no transmitiría.

Una propiedad destacable de *LwF* es que opera exclusivamente sobre los datos de la tarea actual: no requiere *buffer* ni acceso a muestras de tareas anteriores. La señal de destilación se genera evaluando el modelo de referencia sobre los datos nuevos, asumiendo que esas predicciones reflejan el conocimiento previo. Esta característica lo hace atractivo desde el punto de vista de la privacidad y la eficiencia en memoria, pero también constituye su principal limitación: si la distribución de los datos nuevos difiere significativamente de la de las tareas anteriores, las predicciones del modelo de referencia sobre esos datos pueden ser poco informativas o directamente ruidosas, debilitando la señal de destilación y reduciendo la efectividad del método.

II-C4. Contrastive Continual Learning (Co²L)

El cuarto método implementado es **Contrastive Continual Learning** (Co²L), propuesto por Cha et al. [2]. Co²L pertenece a la familia de métodos basados en *rehearsal* y se distingue de los enfoques anteriores en dos aspectos fundamentales. En primer lugar, aprende representaciones mediante un *objetivo contrastivo* en lugar de *entropía cruzada estándar*, lo que produce embeddings más transferibles a tareas futuras y más robustos al olvido catastrófico. En segundo lugar, preserva el conocimiento previo regulando la estructura relacional de las representaciones en lugar de los parámetros del modelo o las predicciones del clasificador. La función de pérdida combinada es:

$$\mathcal{L} = \mathcal{L}_{\text{asym}}^{\text{sup}} + \lambda \cdot \mathcal{L}^{\text{IRD}} \quad (9)$$

donde λ es un hiperparámetro que pondera la contribución de cada término. A continuación se describe cada componente.

II-C4a. Asymmetric Supervised Contrastive Loss.

Durante el entrenamiento de la tarea t , el batch se construye como la unión de muestras de la tarea actual y muestras del *replay buffer*. Co²L introduce una versión asimétrica de la *loss SupCon* descrita en la sección II-B: únicamente las muestras de la tarea actual actúan como anclas, mientras que las muestras del buffer participan exclusivamente como negativos en el denominador. Denotando como $\mathcal{S} \subset \{1, \dots, 2N\}$ el conjunto de índices correspondientes a muestras de la tarea actual, la loss asimétrica se define como:

$$\mathcal{L}_{\text{asym}}^{\text{sup}} = \sum_{i \in \mathcal{S}} \frac{-1}{|\mathcal{P}_i|} \sum_{p \in \mathcal{P}_i} \log \frac{\exp(z_i \cdot z_p / \tau)}{\sum_{k \neq i} \exp(z_i \cdot z_k / \tau)} \quad (10)$$

donde $\mathcal{P}_i = \{j \neq i : y_j = y_i\}$ es el conjunto de positivos respecto al ancla i y τ es la temperatura. La motivación de este diseño asimétrico es doble. Por un lado, evita que el modelo sobreajuste a las pocas muestras del buffer, que constituyen una representación reducida y potencialmente sesgada de las tareas anteriores. Por otro lado, al incluir las muestras del buffer en el denominador, estas actúan como negativos adicionales que enriquecen la señal contrastiva y contribuyen a mantener las representaciones de clases antiguas separadas de las nuevas en el espacio de embeddings.

II-C4b. Instance-wise Relation Distillation (IRD).

El segundo componente de Co²L es la destilación de relaciones instancia a instancia (*IRD*), que preserva la estructura relacional de las representaciones aprendidas por el modelo al final de la tarea anterior. A diferencia de *LwF*, que distila las predicciones del clasificador, *IRD* opera directamente en el espacio de proyección contrastivo $z = (g \circ f)_{\psi}(\tilde{x})$ (el mismo espacio donde se computa la loss asimétrica) y no requiere un clasificador conjunto. Esta elección no es trivial: Cha et al. muestran experimentalmente que distilan en el espacio de proyección $g \circ f$ y no en el espacio de representación f produce mejoras significativas en todos los escenarios evaluados, debido a que la estructura relacional relevante para la loss contrastiva se encuentra en ese espacio y no en el del encoder solo.

Para cada muestra $\tilde{x}_i$ en el batch, se define el *vector de similitud instancia* a instancia como la distribución softmax de sus similitudes con el resto de muestras del batch:

$$p(\tilde{x}_i; \psi, \kappa)_j = \frac{\exp(z_i \cdot z_j / \kappa)}{\sum_{k \neq i} \exp(z_i \cdot z_k / \kappa)}, \quad j \neq i \quad (11)$$

donde $z_i = (g \circ f)_\psi(\tilde{x}_i)$ es la proyección normalizada de la muestra y κ es un hiperparámetro de temperatura. La *loss IRD* cuantifica la discrepancia entre estas distribuciones relacionales del modelo actual ψ y del modelo de referencia ψ^* , fijado al final de la tarea anterior:

$$\mathcal{L}^{\text{IRD}} = \sum_{i=1}^{2N} -p(\tilde{x}_i; \psi^*, \kappa^*) \cdot \log p(\tilde{x}_i; \psi, \kappa) \quad (12)$$

donde κ y κ^* son temperaturas independientes para el modelo actual y de referencia respectivamente. El uso de temperaturas distintas permite controlar por separado la nitidez de ambas distribuciones: κ^* pequeño produce una distribución de referencia más concentrada, actuando como un target más informativo para guiar la destilación.

La intuición detrás de *IRD* es que preservar las relaciones de similitud entre instancias es más robusto que preservar los valores absolutos de las representaciones: aunque las representaciones individuales puedan desplazarse en el espacio de embeddings al aprender nuevas tareas, si su estructura relacional se mantiene, el conocimiento codificado en esas representaciones se conserva.

II-C4c. Evaluación y gestión del buffer.

Al finalizar el entrenamiento de cada tarea, el buffer se actualiza incorporando nuevas muestras mediante *reservoir sampling*, manteniendo una distribución uniforme sobre todas las tareas observadas. Dado que Co^2L es un método de aprendizaje de representaciones y no un esquema de entrenamiento conjunto representación-clasificador, la evaluación requiere entrenar un clasificador lineal adicional al finalizar cada tarea.

II-C5. Experience Replay with Asymmetric Cross-Entropy (ER-ACE)

El quinto método implementado es **Experience Replay with Asymmetric Cross-Entropy (ER-ACE)**, propuesto por Caccia et al. [6]. *ER-ACE* pertenece a la familia de métodos basados en *rehearsal* y opera sobre el espacio de predicciones mediante entropía cruzada estándar, a diferencia de Co^2L que opera sobre representaciones contrastivas. Su contribución principal es una modificación asimétrica de la función de pérdida que separa el tratamiento de las muestras entrantes de la tarea actual y las muestras recuperadas del buffer, abordando un problema de interferencia de gradientes presente en los métodos de replay naïve.

II-C5a. Motivación: interferencia de gradientes en replay naïve.

En Experience Replay estándar (*ER*), la *loss* se calcula conjuntamente sobre el batch de la tarea actual y las muestras del buffer, utilizando entropía cruzada sobre todas las clases:

$$\mathcal{L}^{\text{ER}}(\theta) = \mathcal{L}^{\text{CE}}(\mathcal{B}_t \cup \mathcal{B}_{\text{buf}}) \quad (13)$$

El problema es que al calcular la CE sobre todas las clases simultáneamente, los logits de las clases entrantes y las clases antiguas compiten en el mismo denominador de la softmax. Cuando el modelo actualiza sus pesos para aumentar la probabilidad de las clases nuevas, suprime implícitamente las probabilidades de las clases antiguas — incluso cuando las muestras de esas clases están presentes en el mismo batch. Este fenómeno se conoce como *cross-task softmax interference*.

II-C5b. La solución asimétrica.

ER-ACE resuelve esta interferencia separando la *loss* en dos términos con dominios de clases distintos. Para las muestras de la tarea entrante, la entropía cruzada se calcula restringida exclusivamente a las clases de esa tarea:

$$\mathcal{L}^{\text{in}}(\theta) = -\frac{1}{|\mathcal{B}_t|} \sum_{(x,y) \in \mathcal{B}_t} \log \frac{\exp(z_y/1)}{\sum_{c \in \mathcal{C}_t} \exp(z_c/1)} \quad (14)$$

donde $\mathcal{C}_t$ son las clases de la tarea actual y z_c es el c -ésimo logit del modelo. Al restringir el denominador a $\mathcal{C}_t$, el gradiente de esta *loss* no penaliza los logits de las clases antiguas, eliminando la interferencia cruzada.

Para las muestras del buffer, la entropía cruzada se calcula sobre la unión de las clases antiguas y las clases entrantes:

$$\mathcal{L}^{\text{buf}}(\theta) = -\frac{1}{|\mathcal{B}_{\text{buf}}|} \sum_{(x,y) \in \mathcal{B}_{\text{buf}}} \log \frac{\exp(z_y)}{\sum_{c \in \mathcal{C}_{\leq t}} \exp(z_c)} \quad (15)$$

donde $\mathcal{C}_{\leq t} = \bigcup_{s \leq t} \mathcal{C}_s$ es el conjunto acumulado de todas las clases vistas hasta la tarea t . Este término preserva el conocimiento sobre las clases antiguas sin interferir con el aprendizaje de las nuevas.

La *función de pérdida total* de *ER-ACE* es:

$$\mathcal{L}(\theta) = \alpha_{\text{in}} \cdot \mathcal{L}^{\text{in}}(\theta) + \alpha_{\text{buf}} \cdot \mathcal{L}^{\text{buf}}(\theta) \quad (16)$$

donde α_{in} y α_{buf} son hiperparámetros que ponderan la contribución de cada término.

II-C5c. Gestión del buffer.

Al finalizar el entrenamiento de cada tarea, el buffer se actualiza con muestras de la tarea recién aprendida mediante *reservoir sampling*, garantizando que la distribución del buffer sea uniforme sobre todas las tareas observadas. A diferencia de Co^2L , las muestras en el buffer se almacenan en el espacio de imagen normalizado y se recuperan sin *augmentation* adicional, ya que *ER-ACE* opera sobre predicciones del clasificador y no sobre representaciones contrastivas donde la diversidad de vistas es crítica.

Desde la perspectiva del balance *stability-plasticity*, *ER-ACE* ofrece un mecanismo más directo que *EWC*: en lugar de penalizar implícitamente el olvido mediante regularización de parámetros, lo previene explícitamente reentrenando sobre muestras reales de tareas anteriores. Esto lo hace especialmente efectivo en escenarios Class-IL, donde la competencia entre clases de distintas tareas en el espacio de predicciones es el principal mecanismo de olvido.

III. RESULTADOS

III-A. Especificaciones técnicas e hiperparámetros

Antes de presentar los resultados y su análisis, resulta necesario detallar las principales especificaciones técnicas y los hiperparámetros utilizados en los experimentos. Esta información permite contextualizar las condiciones de entrenamiento y evaluación de los distintos métodos considerados, y facilita una comparación más clara entre los enfoques estudiados.

En los referido a cuestiones de *hardware*, todos las experiencias se llevaron a cabo utilizando una laptop *Dell G15 5530*. En particular, la que cuenta con una NVIDIA GeForce RTX 4060 de 8GB GDDR6, un Intel Core i7-13650HX y 16GB de RAM.

En lo que respecta a los *hiperparámetros*, todos los métodos parten del mismo backbone ResNet-18 como se estableció en secciones anteriores. El pre-entrenamiento se realizó durante 50 épocas con SGD ($lr = 0.5$, momentum = 0.9, weight decay = 10^{-4}), temperatura $\tau = 0.07$ y dimensión de proyección $d = 128$; los primeros 10 épocas corresponden a un calentamiento lineal (*linear warmup*) seguido de un decaimiento coseno. La cabeza lineal de evaluación se entrenó por separado durante 10 épocas ($lr = 1.0$, milestones en épocas 30, 37 y 45 con factor $\gamma = 0.2$).

Para el **Fine-tuning naive**, cada tarea se entrenó durante 50 épocas con SGD ($lr = 0.1$, momentum = 0.9, weight decay = 10^{-4}), sin ningún mecanismo de mitigación del olvido. Para **EWC**, se utilizó la misma cantidad de épocas y momentum, pero con $lr = 0.01$ y un coeficiente de penalización $\lambda_{EWC} = 5.0$ (equivalente a $\lambda = 10$ en la formulación original del paper, donde el factor $1/2$ está absorbido en el coeficiente); la diagonal de la matriz de información de Fisher se estimó sobre un máximo de 100 batches usando el modo `nll_pred`, es decir, tomando el gradiente de la log-probabilidad de la clase predicha por el modelo. Para **LwF**, también se usaron 50 épocas por tarea con $lr = 0.01$, temperatura de destilación $T = 2.0$ y peso de la pérdida de destilación $\alpha = 0.5$.

El método Co^2L se entrenó durante 50 épocas por tarea con SGD ($lr = 0.25$, momentum = 0.9, weight decay = 10^{-4}) y un schedule de warmup lineal (10 épocas) seguido de decaimiento coseno. Los hiperparámetros contrastivos siguen la configuración reportada en el paper para Seq-CIFAR-10: temperatura asimétrica $\tau = 0.5$, temperaturas de destilación $\kappa = 0.2$ y $\kappa^* = 0.01$, y peso de la pérdida IRD $\lambda_{IRD} = 1.0$. El tamaño de batch contrastivo fue de 128 muestras de la tarea actual, con un máximo de 64 muestras del buffer de replay por paso. Al finalizar cada tarea, se entrenó una cabeza lineal sobre el backbone congelado durante 100 épocas ($lr = 1.0$, milestones en épocas 60, 75 y 90, $\gamma = 0.2$) con muestreo balanceado por clase. Finalmente, **ER-ACE** utilizó 50 épocas por tarea con $lr = 0.01$, un batch de replay de 128 muestras, y pesos iguales para la pérdida sobre datos entrantes y datos del buffer ($w_{in} = w_{replay} = 1.0$). En todos los métodos que requieren buffer de replay, se empleó un buffer de capacidad fija $|\mathcal{M}| = 200$ con política de *reservoir sampling*.

III-B. Entrenamiento del backbone y cabeza de clasificación lineal

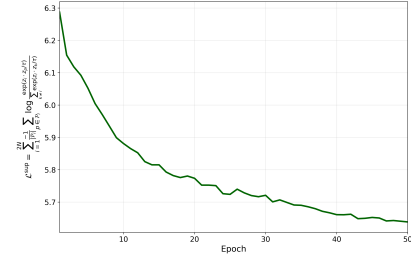


Figura 1. Evolución de la función de pérdida *Supervised Contrastive* (SupCon) durante el pre-entrenamiento de *backbone* (ResNet-18) sobre la *Task 0*.

Tal como se puede apreciar en la figura 1, se pudo entrenar de manera correcta el backbone utilizando la función de pérdida *SupCon*, alcanzando una convergencia estable a lo largo de las 50 épocas. La pérdida disminuye de manera consistente, lo que indica que el modelo está aprendiendo representaciones cada vez más discriminativas para las clases de la tarea inicial. El siguiente paso es analizar cómo evoluciona el espacio latente durante el pre-entrenamiento. Para esto, se proyectaron los embeddings de las imágenes a 2D utilizando la técnica de reducción de dimensionalidad *UMAP* (Uniform Manifold Approximation and Projection). Esta visualización nos permite observar cómo las representaciones de las imágenes se organizan en el espacio latente a medida que el modelo aprende a discriminar entre las clases de la tarea inicial. Tal como

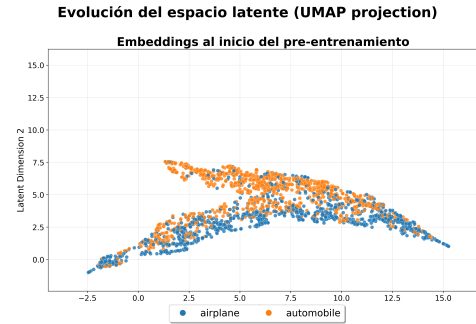


Figura 2. Estado inicial del espacio latente antes de comenzar el entrenamiento.

se puede observar en las figuras 2, 3 y 4, el espacio latente evoluciona significativamente a lo largo del pre-entrenamiento. Al inicio, las representaciones de las imágenes están dispersas sin una estructura clara. A medida que el entrenamiento progresa, comienzan a formarse agrupamientos más definidos, y al finalizar el pre-entrenamiento, las representaciones se organizan en clusters claramente diferenciados por clase. Esta evolución refleja el proceso de aprendizaje del modelo, que va ajustando sus pesos para crear un espacio latente donde las clases sean fácilmente separables.

Y en la figura 5 vemos que el clasificador lineal logró una precisión del 100% en train y aproximadamente 99%

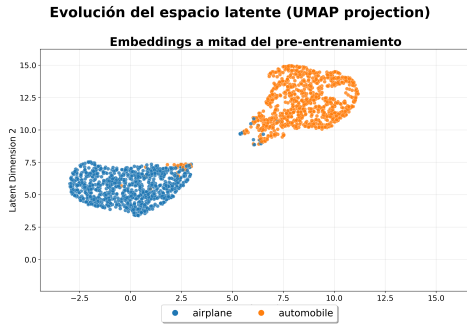


Figura 3. Evolución del espacio latente a mitad del pre-entrenamiento (25 epochs).

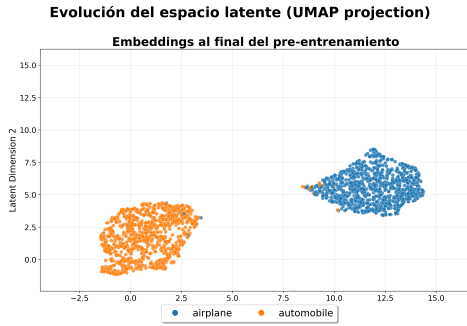


Figura 4. Estado final del espacio latente al finalizar el pre-entrenamiento (50 epochs).

en test, lo que indica que el backbone ha aprendido a generar representaciones altamente discriminativas para las clases de la tarea inicial. Esta alta precisión en el conjunto de prueba sugiere que el modelo ha capturado patrones generales que permiten una buena generalización a datos no vistos.

Luego, se evaluó el desempeño de los distintos métodos de aprendizaje incremental en los escenarios de *Class-IL* y *Task-IL*. La métrica principal utilizada en ambos casos fue la precisión promedio (*accuracy*), definida como el promedio de la precisión obtenida en cada tarea luego de completar el entrenamiento sobre todas las tareas. En *Class-IL*, la precisión se calcula sobre el conjunto acumulado de clases vistas hasta

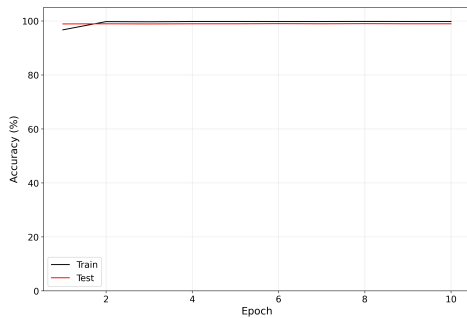


Figura 5. Evolución de la precisión (*accuracy*) durante el entrenamiento de un clasificador lineal sobre las representaciones congeladas del backbone entrenado previamente.

el momento, mientras que en *Task-IL* se utiliza una máscara de tarea (*Task Oracle*) para evaluar únicamente las clases correspondientes a cada tarea. Además, se midió el olvido promedio acumulado mediante la métrica de *Maximum History*, que cuantifica la degradación del rendimiento en tareas previas como la diferencia entre la máxima precisión histórica alcanzada en cada tarea y la precisión obtenida tras entrenar tareas posteriores. En *Task-IL*, esta métrica también se calcula aplicando la máscara de tarea.

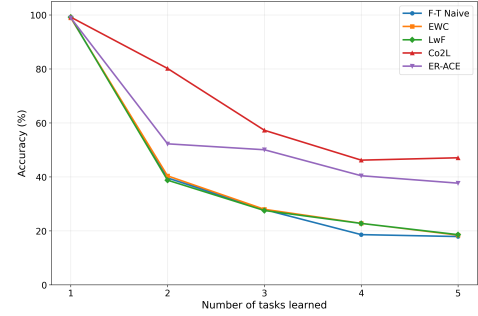


Figura 6. Evolución de la precisión promedio (*accuracy*) en el escenario *Class-Incremental Learning* (Class-IL) de los métodos implementados.

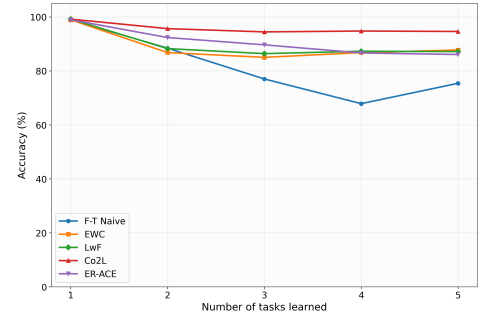


Figura 7. Evolución de la precisión promedio (*accuracy*) en el escenario *Task-Incremental Learning* (Task-IL).

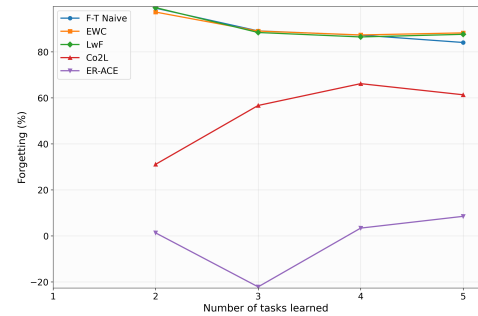


Figura 8. Olvido promedio acumulado en el escenario de incremento de clases (*Class-IL*). Cada punto representa el promedio del olvido de todas las tareas aprendidas hasta ese paso.

A partir de la tabla II y las figuras 6, 7, 8 y 9, se observan diferencias claras entre los escenarios *Class-IL* y *Task-IL*.

En *Task-IL*, todos los métodos excepto el baseline naive superan el 85 % de precisión, reflejando que este escenario es

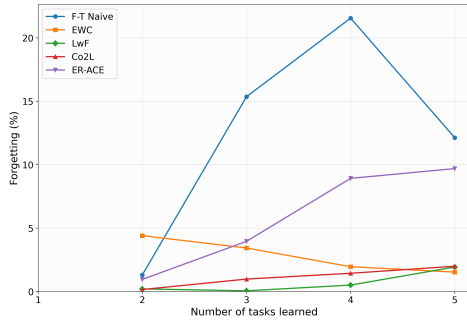


Figura 9. Olvido promedio acumulado en el escenario de incremento de tareas (*Task-IL*).

Tabla II
TABLA COMPARATIVA DE METODOS.

Mtd.	Class-IL	Task-IL	Forgett. (C)	Forgett. (T)	Train
Naive	17.89 %	75.42 %	84.04 %	12.12 %	18 min.
EWC	18.41 %	87.76 %	88.21 %	1.52 %	20 min.
LwF	18.59 %	87.18 %	87.59 %	1.93 %	24 min.
Co2L	47.08 %	94.63 %	61.31 %	1.99 %	101 min.
ER-ACE	37.67 %	86.09 %	8.50 %	9.69 %	30 min.

menos desafiante debido al uso de cabezas separadas por tarea. Co2L obtiene el mejor desempeño (94.63 %), evidenciando la efectividad de su enfoque contrastivo y de destilación para aprender nuevas tareas sin degradar significativamente el conocimiento previo.

En *Class-IL*, la competencia entre clases incrementa la dificultad y se observa mayor variabilidad. El método naive alcanza solo 17.89 %, mientras que EWC y LwF presentan mejoras marginales. ER-ACE mejora notablemente (37.67 %), aunque Co2L logra el mejor resultado (47.08 %), lo que sugiere que su representación latente estructurada facilita la incorporación de nuevas clases y reduce la interferencia.

Respecto al olvido, en *Class-IL* ER-ACE obtiene el menor valor (8.50 %), mostrando la efectividad de su estrategia de replay asimétrico. Co2L también reduce el olvido frente a los métodos clásicos, aunque mantiene un nivel considerable (61.31 %). En *Task-IL*, el olvido es bajo en general (menor al 10 %), consistente con la menor complejidad del escenario.

III-C. Métricas adicionales y análisis complementarios

En el código adjunto y en el *notebook* de Jupyter incluido se pueden encontrar análisis adicionales y visualizaciones complementarias que enriquecen el presente trabajo pero no se incluyen en este informe por limitaciones de espacio. Entre estos análisis se destacan: matrices de confusión normalizadas por método que permiten analizar el patrón de confusiones entre clases en cada enfoque, gráficos heatmap de precisión tarea a tarea para visualizar cómo cada método se desempeña en la intersección de tareas históricas y tareas actuales, y visualizaciones comparativas del espacio de embeddings (reducidas a 2D mediante t-SNE/UMAP) para todos los métodos, mostrando cómo cada estrategia organiza y estructura las representaciones aprendidas en el espacio latente a través de las distintas clases. Estas visualizaciones permiten comprender de

forma intuitiva el compromiso entre plasticidad y estabilidad que caracteriza a cada método.

IV. CONCLUSIONES

En este trabajo práctico se estudió el problema del aprendizaje continuo en clasificación de imágenes sobre *Seq-CIFAR-10*, comparando distintos enfoques representativos bajo los escenarios *Class-IL* y *Task-IL*. Los resultados evidencian una diferencia marcada entre ambos escenarios: mientras que en *Task-IL* la mayoría de los métodos alcanza precisiones elevadas y el olvido se mantiene bajo, en *Class-IL* el desempeño cae drásticamente debido a la competencia directa entre clases de tareas distintas, lo que expone con mayor claridad el fenómeno de olvido catastrófico.

En particular, el baseline naive mostró un rendimiento muy limitado en *Class-IL*, confirmando que el fine-tuning secuencial es insuficiente en contextos continuos. Métodos clásicos como EWC y LwF solo lograron mejoras marginales, sugiriendo que la regularización de parámetros o la destilación por sí solas no alcanzan para enfrentar la interferencia entre tareas en este escenario. En contraste, los métodos más modernos obtuvieron mejoras sustanciales: Co2L logró la mayor precisión en ambos escenarios, indicando que el aprendizaje contrastivo permite construir representaciones más robustas y transferibles. Por otro lado, ER-ACE se destacó por reducir significativamente el olvido en *Class-IL*, mostrando que estrategias basadas en replay y separación asimétrica de la pérdida son efectivas para preservar conocimiento previo.

En conclusión, los resultados muestran que la dificultad del aprendizaje continuo depende fuertemente del escenario de evaluación, y que las técnicas modernas basadas en representaciones contrastivas y replay superan ampliamente a los métodos clásicos en términos de precisión y estabilidad. Esto refuerza la importancia de diseñar mecanismos explícitos para controlar la interferencia entre tareas y mantener representaciones consistentes a lo largo del entrenamiento incremental.

REFERENCIAS

- [1] P. Khosla et al., "Supervised Contrastive Learning," *arXiv preprint arXiv:2004.11362*, 2020.
- [2] H. Cha, J. Lee, and J. Shin, "Co2L: Contrastive Continual Learning," *arXiv preprint arXiv:2106.14413*, 2021.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *arXiv preprint arXiv:1512.03385*, 2015.
- [4] J. Kirkpatrick et al., "Overcoming Catastrophic Forgetting in Neural Networks," *arXiv preprint arXiv:1612.00796*, 2017.
- [5] Z. Li and D. Hoiem, "Learning without Forgetting," *arXiv preprint arXiv:1606.09282*, 2017.
- [6] L. Caccia, R. Aljundi, N. Asadi, T. Tuytelaars, J. Pineau, and E. Belilovsky, "New Insights on Reducing Abrupt Representation Change in Online Continual Learning," *arXiv preprint arXiv:2104.05025*, 2021.